Lab 3.2: Customizing the Common Dialog Class

In this lab, you will create a customized File Open template, create a dialog class, and add custom controls and handlers for the newly created dialog class.

To see a demonstration of the lab solution, click this icon.
(CD-ROM plays the demonstration, "Lab 3.2 Demonstration.")

Estimated time to complete this lab: **75 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch03\Lab3.2.

The solution code for this lab is located in the folder \Labs\Ch03\Lab3.2.

Objectives

After completing this lab, you will be able to:

- Create a template to expand the functionality of a common dialog box.
- Create a class that expands the functionality of a common dialog box.
- Respond to messages sent by a common dialog box.
- Stream data into a rich edit control.
- Incorporate an expanded common dialog box into an existing application.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter:

Exercise 1: Creating a Customized File Open Template

In this exercise, you will create a child dialog box of a common (parent) dialog box. The child dialog box will inherit all the controls of the parent dialog box.

Exercise 2: Customizing the Dialog Class

In this exercise, you will use the ClassWizard to create the dialog class and add custom controls and handlers for the newly created dialog class.

Exercise 3: Implementing Dialog Class Handlers

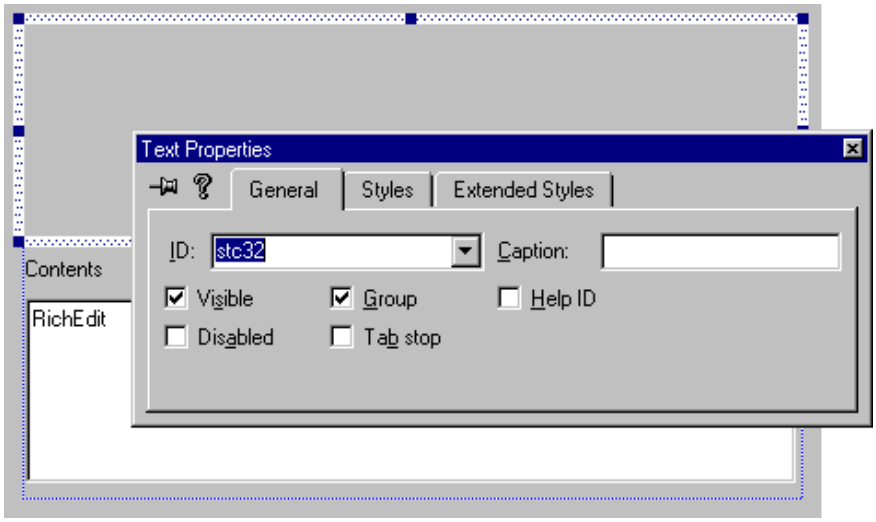
In this exercise, you will implement the handlers created in Exercise 2 and their supporting functions in the dialog class.

60 Exercise 1: Creating a Customized File Open Template

The code that forms the starting point for this exercise is in \Labs\Ch03\Lab3.2\Ex01.

In this exercise, you will create a child dialog box of a common (parent) dialog box. The child dialog box will inherit all the controls of the parent dialog box.

The following illustration shows the completed template.



▶ **Create a new child dialog box of the common dialog box** File Open

☐ 1. On the **Insert** menu, click **Resource**, or press CTRL+R.

☐ 2. In the **Insert Resource** dialog box, click **Dialog** and click **New**.

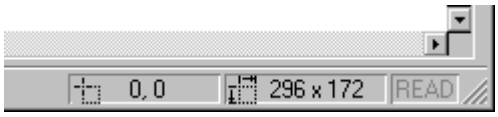
☐ 3. Open the **Dialog Properties** dialog box and set the ID to IDD_FILEOPEN_EX.

☐ 4. On the **Styles** tab, set **Style** to **Child** and **Border** to **None**. Clear the default **Title bar** check box if it is selected. Select **Clip siblings** and **Clip children**.

☐ 5. On the **More Styles** tab, select **Visible**, **3D-look**, and **Control**.

When you select **Control**, the parent dialog box (**File Open**) can tab into the child (IDD_FILEOPEN_EX).

☐ 6. Resize the dialog box to 296 by 172. The size appears in the right pane of the status bar, as shown in the following illustration.



☐ 7. Delete both the **OK** and **Cancel** buttons.

☐ 8. Draw a static text control from the position 7,7 sized to 276 by 71. Set the ID to **stc32** so that Windows® will draw the **File** dialog box within it. Clear its caption.

Notes

☐ 9. Draw a static text control from the position 7,84 sized to 31 by 8. Set the Caption property of the static control to **Contents**.

☐ 10. Draw a rich edit control from the position 7,98 sized to 276 by 63.

☐ 11. Set the ID property of the rich edit control to IDC_EDIT_CONTENTS.

☐ 12. Clear the **Tabstop** option.

☐ 13. Save Diff.rc.

☐ 14. Directly edit Diff.rc and set the style of the rich edit control to ES_MULTILINE | WS_BORDER | WS_VSCROLL | WS_HSCROLL.

This style enables the user to scroll through the file.

☐ 15. Open Resource.h and remove the definition of **stc32** added by the dialog editor.

(CD-ROM displays a lab hint.)

☐ 16. Save Resource.h.

☐ 17. The constant **stc32** is defined in dlgs.h, so include dlgs.h in Diff.rc.

The solution code for this exercise is located in the folder \Labs\Ch03\Lab3.2\Ex01\Solution.

Notes

60 Exercise 2: Customizing the Dialog Class

Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch03\Lab3.2\Ex02.

In this exercise, you will use ClassWizard to create the dialog class and add custom controls and handlers for the newly created dialog class.

▶ Use ClassWizard to create the dialog class

In this procedure, you will invoke the ClassWizard to create the dialog class.

(CD-ROM displays the following tip.)

To have ClassWizard automatically display the **Adding a Class** dialog box, open ClassWizard from the dialog editor session where you just created the new dialog resource.d.

☐ 1. Run the dialog editor on the IDD_FILEOPEN_EX dialog resource. Be sure that your dialog box template window is the active child window.

☐ 2. Invoke **ClassWizard**. It should automatically display the **Adding A Class** dialog box. (If for some reason this dialog box does not appear, click the **Add Class** button).

☐ 3. Click **Create a new class** option, and click **OK**. The **Create New Class** dialog box appears.

☐ 4. In the **Name** box, type **CFileOpenEx** for the name of the associated C++ dialog class.

☐ 5. In the **Base Class** box, select **CDialog**.

The Dialog ID should already be set to IDD_FILEOPEN_EX.

ClassWizard provides different styles of automation for dialog boxes derived from **CCommonDialog** and **CDialog**. To use ClassWizard to best effect, begin deriving from **CDialog,** and later derive from **CFileDialog**. Do this in ClassWizard and modify the class in the editor.

☐ 6. Click **OK**.

▶ Add a member variable to the dialog class

In this procedure, you will create a member variable for the new **CFileOpenEx** class.

- Right-click **CFileOpenEx** in the ClassView window. Add the following protected member variable:

```
CRichEditCtrl   m_RichEdit
```

Notes

▶ Create handlers for the dialog class

Using ClassWizard, add a handler for **DoModal**. You will need to add the handler for **OnFileNameChange** manually.

☐ 1. Invoke ClassWizard for the Diff project.

☐ 2. Click the **Message Maps** tab in ClassWizard.

☐ 3. Create a handler for a dialog message.

a. In the **Class Name** drop-down list box, click the dialog class **CFileOpenEx**.

b. In the **Object ID** list box, click **CFileOpenEx** as the class name.

c. In the **Messages** list box, click **DoModal**.

d. Click **Add Function** and accept the default function name.

The function name will be displayed in the **Member Functions** list box.

☐ 4. Click **OK**.

▶ Create a handler for a function not listed in ClassWizard

When you change the selected file in the Explorer-style list in the **File** dialog box, a CDN_SELCHANGE message is sent to the dialog box itself. (For more information about this message, see "Open and Save As Dialog Box Messages and Notifications" in the MSDN™ Library Visual Studio™ 6.0.) You will need to examine the source code for the **CFileDialog** class to determine how this message is handled.

☐ 1. In DlgFile.cpp, in the MFC source code, examine the **CFileDialog::OnNotify** handler:

```
BOOL CFileDialog::OnNotify(WPARAM wParam, LPARAM lParam, LRESULT* pResult)
{
    ...
    switch(pNotify->hdr.code)
    {
    ...
    case CDN_SELCHANGE:
        OnFileNameChange();
        return TRUE;
    ...
}
```

Note that when a CDN_SELCHANGE message is received, the dialog invokes **OnFileNameChange** procedure.

☐ 2. Add the **OnFileNameChange** to the **CFileOpenEx** class as a protected function as follows:

```
virtual void OnFileNameChange()
```

☐ 3. Build your project.

The solution code for this exercise is located in the folder \Labs\Ch03\Lab3.2\Ex02\Solution.

Notes

60 Exercise 3: Implementing Dialog Class Handlers

Continue with the files you created in Exercise 2 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch03\Lab3.2\Ex03.

CDialogFileEx has two primary entry points:

- **DoModal**
- **OnFileNameChange**

In this exercise, you will implement each of these handlers and their supporting functions.

▶ Change the derivation of CFileOpenEx to CFileDialog

The **CFileDialog** class encapsulates the Windows common file dialog box. Common file dialog boxes provide an easy way to implement **File Open** and **File Save As** dialog boxes (as well as other file-selection dialog boxes) in a manner consistent with Windows standards. You can derive your own dialog class from **CFileDialog** and write a constructor for it.

☐ 1. Open FileOpenEx.h.

☐ 2. Change the parent class to **CFileDialog** as follows:

```
class CFileOpenEx : public CFileDialog
```

☐ 3. Save FileOpenEx.h.

☐ 4. Open FileOpenEx.cpp and scroll to the **CFileOpenEx** constructor. Convert the default constructor code for **CFileDialog** from:

```
CFileOpenEx::CFileOpenEx(CWnd* pParent /*=NULL*/)
    : CDialog(CFileOpenEx::IDD, pParent)
{
    //{{AFX_DATA_INIT(CFileOpenEx)
        // NOTE: the ClassWizard will add member initialization here
    //}}AFX_DATA_INIT
}
```

To a constructor as follows:

```
CFileOpenEx::CFileOpenEx(   LPCTSTR lpszDefExt /*= NULL*/,
                            LPCTSTR lpszFileName /*= NULL*/,
                            DWORD dwFlags /*= OFN_HIDEREADONLY*/,
                            LPCTSTR lpszFilter /*= NULL*/,
                            CWnd* pParentWnd /*= NULL*/)
            : CFileDialog(TRUE, lpszDefExt, lpszFileName, dwFlags,
lpszFilter, pParentWnd)
{

    //{{AFX_DATA_INIT(CFileOpenEx)
        // NOTE: ClassWizard will add member initialization here
    //}}AFX_DATA_INIT
}
```

☐ 5. Convert all occurrences of **CDialog** in FileOpenEx.cpp to **CFileDialog**.

☐ 6. Save FileOpenEx.cpp.

Notes

☐ 7. Open FileOpenEx.h.

☐ 8. Change the default constructor declaration from:

```
CFileOpenEx(CWnd* pParent = NULL);   // standard constructor
```

To reflect the new constructor declaration:

```
CFileOpenEx(LPCTSTR lpszDefExt = NULL,
            LPCTSTR lpszFileName = NULL,
            DWORD dwFlags = OFN_HIDEREADONLY,
            LPCTSTR lpszFilter = NULL,
            CWnd* pParentWnd = NULL);
```

☐ 9. Save FileOpenEx.h.

▶ Set up DDX

Because a rich edit control is a custom control from the resource editor, you need to set up dialog data exchange (DDX) yourself.

☐ 1. Add a call to **DDX_Control** in **DoDataExchange** as follows:

```
DDX_Control(pDX, IDC_EDIT_CONTENTS, m_RichEdit);
```

☐ 2. Save FileOpenEx.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CFileOpenEx::DoDataExchange(CDataExchange* pDX)
{
    CDialog::DoDataExchange(pDX);
    //{{AFX_DATA_MAP(CFileOpenEx)
    //}}AFX_DATA_MAP
        DDX_Control(pDX, IDC_EDIT_CONTENTS, m_RichEdit);
}
```

▶ Link the dialog box to the template in DoModal

DoModal associates the template you have created with the **CFileDialog** common dialog box and then calls **CFileDialog::DoModal**. ClassWizard provides an **enum** IDD that isolates your code from changes you may make in your dialog box resource.

☐ 1. Call the function **SetTemplate** as follows:

```
SetTemplate(NULL, MAKEINTRESOURCE(IDD_FILEOPEN_EX));
```

☐ 2. Save FileOpenEx.cpp.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
int CFileOpenEx::DoModal()
{
    SetTemplate(NULL, MAKEINTRESOURCE(IDD_FILEOPEN_EX));

    return CFileDialog::DoModal();

}
```

▶ **Implement OnFileNameChange**

In this procedure, you will display a file when the user selects it in the pane.

☐ 1. Get the path name from the **File** dialog box, and check to see that it is not blank:

```
CString FileName = GetPathName();
if(FileName.GetLength())
{
```

☐ 2. Check to see whether the file exists:

```
OFSTRUCT    of;
if(OpenFile(FileName, &of, OF_EXIST) != HFILE_ERROR)
```

☐ 3. You will implement **PreviewContents** to show the file in the rich edit control later in this exercise. At this point, simply call **PreviewContents** as follows:

```
PreviewContents(FileName);
```

☐ 4. Save FileOpenEx.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CFileOpenEx::OnFileNameChange()
{
    CString FileName = GetPathName();
    if(FileName.GetLength())
    {
        OFSTRUCT    of;
        if(OpenFile(FileName, &of, OF_EXIST) != HFILE_ERROR)
        {
            PreviewContents(FileName);
        }
    }
}
```

Notes

▶ Implement a function to load the file into the preview

CRichEditCtrl has stream-oriented input and output provided by **StreamIn** and **StreamOut**. You will use **StreamIn** to load the preview from the selected file. **StreamIn** uses a callback function to do the actual file processing, and you will specify that function as part of an EDITSTREAM structure.

☐ 1. Add **PreviewContents** to the **CFileOpenEx** class as a protected member function:

```
void     PreviewContents(LPCSTR lpszFilespec);
```

☐ 2. In the function **PreviewContents**, open the file specified in lpszFilespec as follows:

```
CFile File(lpszFilespec, CFile::modeRead);
```

☐ 3. Declare an EDITSTREAM structure. Set the cookie to **CFile**, clear the error return, and set the callback to OpenCallback:

```
EDITSTREAM es;
es.dwCookie = (DWORD)&File;
es.dwError=0;
es.pfnCallback=OpenCallback;
```

☐ 4. Send the **StreamIn** message, specifying that the data is to be read as plain text:

```
m_RichEdit.StreamIn(SF_TEXT, es);
```

☐ 5. Save FileOpenEx.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CFileOpenEx::PreviewContents(LPCSTR lpszFilespec)
{
    CFile File(lpszFilespec, CFile::modeRead);

    EDITSTREAM      es;
    es.dwCookie = (DWORD)&File;
    es.dwError      = 0;
    es.pfnCallback  = OpenCallback;

    m_RichEdit.StreamIn(SF_TEXT, es);
}
```

▶ Implement the callback function to read the file (OpenCallback)

The callback specified in the EDITSTREAM structure does the work of reading the data stream into a buffer that **CRichEditCtrl** can process.

☐ 1. Before your implementation of **PreviewContents**, declare **OpenCallback** in FileOpenEx.cpp.

An EDITSTREAM callback takes four parameters: the DWORD parameter sent by **StreamIn**, a buffer to receive the stream, the size of the buffer, and a buffer to receive the actual count of bytes read.

```
DWORD CALLBACK OpenCallback(DWORD   dwCookie,
                            LPBYTE  pbBuff,
                            LONG    cb,
                            LONG    *pcb )
{
```

☐ 2. Cast dwCookie to a **CFile** pointer.

Notes

☐ 3. Read cb bytes into pbBuff.

☐ 4. Return to indicate that you have processed the call.

☐ 5. Save FileOpenEx.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
DWORD CALLBACK OpenCallback(DWORD   dwCookie,
                            LPBYTE  pbBuff,
                            LONG    cb,
                            LONG    *pcb )
{
    CFile   *pFile = (CFile *)dwCookie;

    *pcb = pFile->Read(pbBuff, cb);
    return 0;
}
```

▶ Enable CDialogFileEx

One of the advantages of maintaining a good object-oriented development style is the ease of adding, removing, or changing components. This is demonstrated in the ease of incorporating the **CFileOpenEx** class into the ShowDiff application. You only need to include the class header and change two constructor calls to enable **CFileOpenEx**.

☐ 1. Open Dlgopenf.cpp.

☐ 2. Include the **CDialogFileEx** header:

```
#include "fileopenex.h"
```

☐ 3. In **OnButtonFile1Browse** handler, replace the **CFileDialog** declarations with **CFileOpenEx** as follows:

```
CFileDialog dlg(TRUE);
//becomes =>
CFileOpenEx dlg;
```

☐ 4. Repeat Step 3 for **OnButtonFile2Browse**.

☐ 5. Save Dlgopenf.cpp.

▶ Build and test the application

- Build your application. Click **Open** and then click **Browse** to select a file. View the contents of a file in the **Contents** box.

The solution code for this exercise is located in the folder \Labs\Ch03\Lab3.2\Ex03\Solution.

Notes